

HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG
KHOA CÔNG NGHỆ THÔNG TIN



BÁO CÁO BÀI TẬP LỚN

**Đề tài: Nhận dạng chữ viết tay và hình dạng đơn giản
bằng mạng neural**

Giảng viên hướng dẫn	Phạm Hoàng Việt
Nhóm lớp	02
Nhóm bài tập lớn	17
Sinh viên thực hiện	Đỗ Đức Cảnh - B22DCCN086 Trần Quang Huy - B22DCCN397 Trần Quang Huy - B22DCCN398

Hà Nội, 11/2025

Đóng góp nhóm

STT	Mã sinh viên	Họ Tên	Đóng góp
1	B22DCCN086	Đỗ Đức Cảnh	35%
3	B22DCCN397	Trần Quang Huy	35%
4	B22DCCN398	Trần Quang Huy	30%

MỤC LỤC

PHẦN 1. TỔNG QUAN ĐỀ TÀI	1
PHẦN 2: BỘ DỮ LIỆU VÀ CÔNG NGHỆ SỬ DỤNG	2
2.1. Bộ dữ liệu	2
2.2. Công nghệ sử dụng	6
PHẦN 3: TIỀN XỬ LÝ DỮ LIỆU	10
3.1. Chuyển đổi ảnh về thang xám (Grayscale Conversion)	10
3.2. Chuẩn hóa pixel về khoảng [0, 1] (Normalization)	10
3.3. Tăng cường tương phản (Histogram Equalization / CLAHE)	10
3.4. Khử nhiễu ảnh (Denoising)	11
3.5. Phân ngưỡng (Thresholding)	12
3.6. Phép hình thái học (Morphological Operations)	13
3.7. Phát hiện biên (Canny Edge Detection)	14
3.8. Connected Component Analysis	14
3.9. Convex Hull	14
3.10. Tìm bounding box & cropping	14
3.11. Resize giữ tỷ lệ (Aspect Ratio Preserving)	15
3.12. Căn giữa hình vào canvas	16
3.13. Làm mượt cuối (Final Smoothing)	16
3.14. Chuẩn hóa đầu ra (Final Normalization)	16
PHẦN 4: KIẾN TRÚC MÔ HÌNH CNN	17
4.1. Tổng quan kiến trúc CNN	17
4.2. Kiến trúc chi tiết mô hình CNN cho MNIST	17
4.3. Giải thích từng nhóm tầng và ý nghĩa	19
4.4. Mối liên hệ giữa kiến trúc CNN và dữ liệu MNIST	21
PHẦN 5: KẾT QUẢ HUẤN LUYỆN MÔ HÌNH	22
5.1. Nhận dạng chữ số MNIST	22
5.1.1. Thiết lập huấn luyện	22
5.1.2. Kết quả định lượng trên tập kiểm tra	23
5.1.3. Đồ thị quá trình huấn luyện (Loss & Accuracy)	24
5.1.4. Nhận xét & đánh giá	25
5.2. Nhận dạng hình học	26
5.2.1. Tổng quan mô hình & dữ liệu	26
5.2.2. Biểu đồ quá trình huấn luyện	26
5.2.3. Bảng kết quả huấn luyện	27
5.2.4. Phân tích chi tiết kết quả	27
5.2.5. Đánh giá độ tin cậy của mô hình	28
PHẦN 6: KẾT QUẢ DEMO VÀ HƯỚNG PHÁT TRIỂN	29
6.1. Demo nhận dạng ký tự	29
6.2. Hướng phát triển	35

PHẦN 1. TỔNG QUAN ĐỀ TÀI

Trong thời đại dữ liệu số và trí tuệ nhân tạo phát triển mạnh mẽ, các kỹ thuật nhận dạng hình ảnh ngày càng được ứng dụng rộng rãi trong thực tế, từ hệ thống phân loại ảnh, xe tự lái, đến nhận dạng chữ viết tay và xử lý biểu mẫu. Trong đó, **nhận dạng chữ số viết tay** là một trong những bài toán kinh điển của lĩnh vực Thị giác máy tính (Computer Vision) và Học sâu (Deep Learning).

Bài toán này không chỉ mang giá trị thực tiễn cao mà còn đóng vai trò như một “**bài tập nền tảng**” giúp nghiên cứu và đánh giá khả năng học đặc trưng của các mô hình mạng nơ-ron, đặc biệt là **mạng nơ-ron tích chập – Convolutional Neural Network (CNN)**.

- Tại sao cần nhận dạng chữ số viết tay?

Nhận dạng chữ viết tay được ứng dụng trong nhiều hệ thống hiện đại như:

- Đọc và phân loại số trên giấy tờ, hóa đơn, phiếu bầu cử
- Nhập liệu tự động cho biểu mẫu
- Xử lý zip-code trong bưu điện
- Nhận dạng ký tự trên séc ngân hàng
- Giao diện người dùng thông minh

Do chữ viết tay của con người rất đa dạng, méo, lệch và nhiều phong cách khác nhau — nên đây là một bài toán thách thức.

- Mục tiêu của đề tài:

Trong báo cáo này, nhóm em tập trung xây dựng và đánh giá một mô hình CNN để:

- Nhận dạng chính xác chữ số viết tay từ **0 đến 9** và các chữ a-z
- Huấn luyện trên bộ dữ liệu **MNIST** gồm 70.000 ảnh
- Áp dụng **Data Augmentation** nhằm tăng độ bền mô hình
- Theo dõi quá trình huấn luyện thông qua biểu đồ Accuracy và Loss
- Đánh giá mô hình trên tập test với độ chính xác cao

- Phạm vi thực hiện:

- Ảnh đầu vào là ảnh đơn lẻ.
- Chỉ tập trung vào chữ số (0–9) và ký tự a-z và các loại hình học cơ bản.
- Không xử lý video, không nhận dạng nhiều đối tượng trong một ảnh.
- Dữ liệu chủ yếu từ MNIST, tập tự thu thập và dataset hình học tự tạo.

- Ý nghĩa của đề tài

Việc xây dựng thành công mô hình không chỉ giúp nhóm hiểu sâu hơn về:

- Kiến trúc CNN
- Cách xử lý dữ liệu ảnh
- Quá trình huấn luyện mô hình học sâu
- Ứng dụng thực tế của mạng nơ-ron

mà còn là tiền đề để mở rộng sang các bài toán nâng cao hơn như:

- Nhận dạng chữ viết tay liên tục (sequence)
- OCR nâng cao
- Nhận dạng ký tự tiếng Việt
- Nhận dạng hình học hoặc ký hiệu toán học

PHẦN 2: BỘ DỮ LIỆU VÀ CÔNG NGHỆ SỬ DỤNG

2.1. Bộ dữ liệu

Trong đề tài này, nhóm sử dụng **ba loại bộ dữ liệu khác nhau**, mỗi loại phục vụ một bài toán nhận dạng riêng: chữ số, chữ cái và hình học. Việc kết hợp ba dạng dữ liệu giúp mô hình được đánh giá toàn diện hơn, từ bài toán đơn giản (chữ số) đến phức tạp (chữ cái) và trừu tượng (hình học).

2.1.1. Bộ dữ liệu chữ số viết tay MNIST

MNIST (Modified National Institute of Standards and Technology) là bộ dữ liệu kinh điển trong lĩnh vực nhận dạng chữ viết tay. Đây là bộ dữ liệu cơ sở để kiểm tra khả năng học đặc trưng của các mô hình CNN.



Bộ dữ liệu MNIST

Thông tin bộ dữ liệu

- Tổng số mẫu: **70.000 ảnh**
 - **60.000 ảnh huấn luyện**
 - **10.000 ảnh kiểm tra**
- Định dạng:
 - Kích thước: **28×28 pixels**

- Loại ảnh: **grayscale**
- Mỗi ảnh chứa **một chữ số viết tay từ 0 đến 9**
- Dữ liệu đã được căn giữa và xử lý cơ bản, giúp việc train nhanh và ổn định.

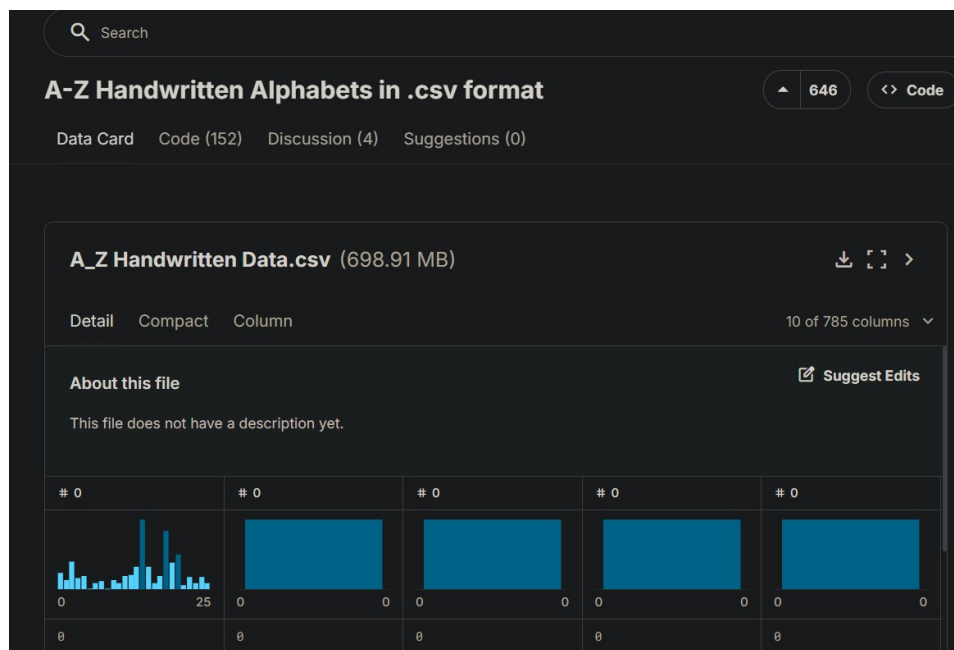
Đặc điểm nổi bật

- Đa dạng nét viết tay từ nhiều người khác nhau.
- Ít nhiễu → phù hợp để xây dựng mô hình CNN cơ bản.
- Là benchmark chuẩn cho rất nhiều mô hình nhận dạng ảnh.

MNIST được sử dụng làm **bộ dữ liệu chính** cho mô hình nhận dạng chữ số của nhóm.

2.1.2. Bộ dữ liệu chữ cái viết tay A–Z (Kaggle Handwritten Alphabets Dataset)

Dữ liệu được nhóm sử dụng là bộ **Handwritten Alphabet Dataset** từ Kaggle – một tập dữ liệu lớn gồm các chữ cái viết tay, được tạo ra để phục vụ bài toán OCR và các mô hình nhận dạng chữ.



Bộ dữ liệu từ Kaggle

Thông tin bộ dữ liệu

- Tổng số ảnh: **khoảng 372.000 mẫu**
- Số lớp: **26 lớp**, tương ứng với 26 chữ cái **A → Z**

- Kích thước ảnh: **28×28 hoặc 32×32 pixels**, grayscale
(tùy phiên bản Kaggle mà nhóm đã sử dụng, nhưng đều phù hợp với CNN)

Đặc điểm dữ liệu

- Mỗi ký tự có nhiều kiểu viết:
 - In hoa, in thường
 - Chữ nghiêng
 - Nét thanh – nét đậm
- Có nhiều trường hợp dễ gây nhầm lẫn:
 - **C ↔ E**
 - **I ↔ L**
 - **O ↔ Q**
 - **U ↔ V**
- Quy mô dữ liệu lớn hơn MNIST rất nhiều → mô hình học phức tạp hơn.

Lý do lựa chọn

- Tăng độ khó của đề tài so với chỉ dùng MNIST.
- Phù hợp với mục tiêu mở rộng bài toán sang **OCR ký tự tiếng Anh**.
- Bộ dữ liệu Kaggle mang tính thực tế cao, gần với bài toán nhận dạng chữ viết tay đời thật.

2.1.3. Bộ dữ liệu hình học tự tạo (Triangle – Rectangle – Circle)

Mục tiêu

Ngoài việc nhận dạng chữ, nhóm xây dựng thêm mô hình CNN để nhận dạng **3 loại hình học cơ bản**:

- **Circle** (Hình tròn)
- **Rectangle** (Hình chữ nhật)
- **Triangle** (Hình tam giác)

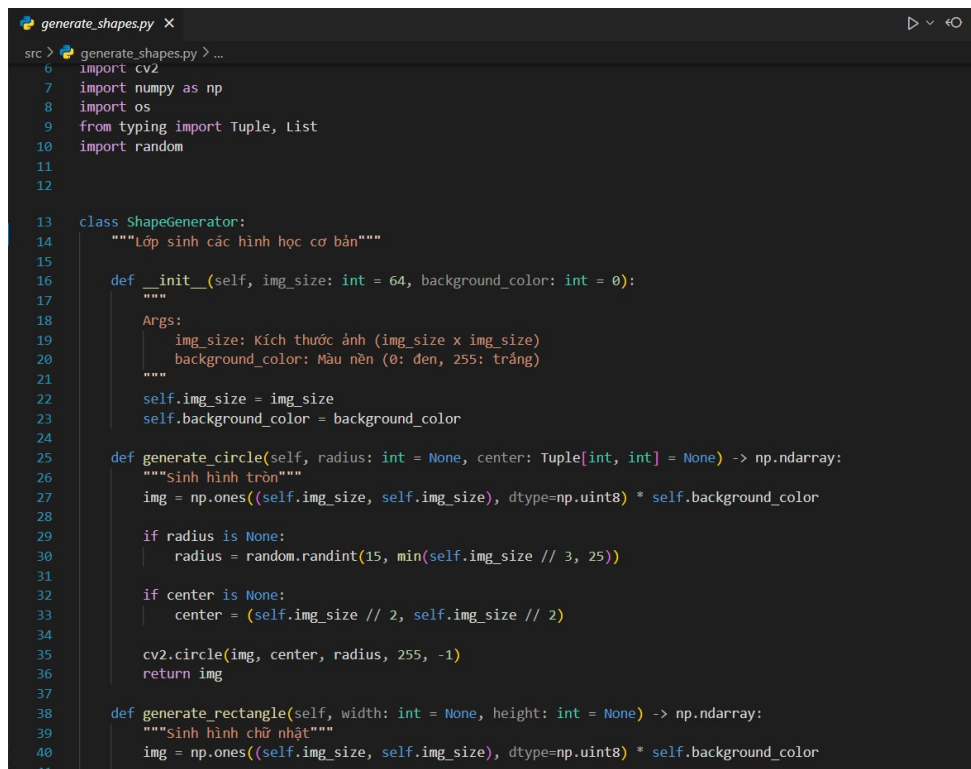
Bài toán này giúp đánh giá khả năng CNN trong việc học đặc trưng hình dạng như:

- Số cạnh
- Góc nhọn/góc vuông
- Độ cong của đường biên

Cách xây dựng bộ dữ liệu

Nhóm tạo dữ liệu bằng script Python (generate_shapes.py) với các đặc điểm:

- **Tự động sinh ảnh** bằng thư viện vẽ hình (cv2, PIL)
- Kích thước ảnh: **64×64 grayscale**
- Số mẫu/lớp: **tùy cấu hình (ví dụ: 1.000 ảnh/lớp)**
- Áp dụng biến đổi dữ liệu khi tạo:
 - Xoay hình một góc ngẫu nhiên
 - Dịch vị trí lên/xuống/trái/phải
 - Biến dạng nhỏ (shear)
 - Thay đổi độ dày hoặc bán kính



```

generate_shapes.py x
src > generate_shapes.py > ...
6 import cv2
7 import numpy as np
8 import os
9 from typing import Tuple, List
10 import random
11
12
13 class ShapeGenerator:
14     """Lớp sinh các hình học cơ bản"""
15
16     def __init__(self, img_size: int = 64, background_color: int = 0):
17         """
18         Args:
19             img_size: Kích thước ảnh (img_size x img_size)
20             background_color: Màu nền (0: đen, 255: trắng)
21         """
22         self.img_size = img_size
23         self.background_color = background_color
24
25     def generate_circle(self, radius: int = None, center: Tuple[int, int] = None) -> np.ndarray:
26         """Sinh hình tròn"""
27         img = np.ones((self.img_size, self.img_size), dtype=np.uint8) * self.background_color
28
29         if radius is None:
30             radius = random.randint(15, min(self.img_size // 3, 25))
31
32         if center is None:
33             center = (self.img_size // 2, self.img_size // 2)
34
35         cv2.circle(img, center, radius, 255, -1)
36         return img
37
38     def generate_rectangle(self, width: int = None, height: int = None) -> np.ndarray:
39         """Sinh hình chữ nhật"""
40         img = np.ones((self.img_size, self.img_size), dtype=np.uint8) * self.background_color
41

```

Hàm sinh dữ liệu hình học

Đặc điểm dữ liệu

- Không phải dữ liệu viết tay → ít nhiễu hơn MNIST và Kaggle

- Dễ phân biệt nhờ đặc trưng hình học rõ ràng
- Tuy nhiên vẫn có độ khó nhất định khi hình bị xoay hoặc lệch
- Phù hợp để kiểm tra mô hình có “học đúng đặc trưng hình dạng” hay không

2.1.4. Tóm tắt so sánh 3 bộ dữ liệu

Thuộc tính	MNIST (Số)	Kaggle A-Z (Chữ cái)	Shapes tự tạo
Loại dữ liệu	Chữ số viết tay	Chữ cái viết tay	Hình học
Số lớp	10	26	3
Kích thước	28×28×1	28×28×1 hoặc 32×32×1	64×64×1
Độ khó	Trung bình	Khó hơn	Dễ – trung bình
Nguồn	Chuẩn MNIST	Kaggle	Tự tạo bằng script
Nhiều	Thấp	Trung bình	Thấp
Biến dạng	Nhiều kiểu nét	Rất đa dạng, dễ nhầm	Xoay, dịch, ít nhiều

2.2. Công nghệ sử dụng

Trong đề tài này, nhóm sử dụng nhiều công nghệ, thư viện và công cụ trong lĩnh vực Trí tuệ nhân tạo, Học sâu và Xử lý ảnh. Các công nghệ này kết hợp tạo thành một pipeline hoàn chỉnh: tải dữ liệu → tiền xử lý → xây dựng mô hình CNN → huấn luyện → đánh giá.

2.2.1. Python – Ngôn ngữ lập trình chính

Toàn bộ hệ thống được xây dựng bằng **Python**, nhờ vào các ưu điểm:

- Hệ sinh thái AI/Deep Learning phong phú
- Cú pháp đơn giản, dễ đọc
- Tích hợp tốt với thư viện TensorFlow, NumPy, OpenCV
- Được cộng đồng nghiên cứu sử dụng rộng rãi

Python là nền tảng chính cho việc phát triển và huấn luyện mô hình CNN.

2.2.2. TensorFlow & Keras – Framework chính cho Deep Learning

- TensorFlow

Là framework học sâu mạnh mẽ do Google phát triển, chịu trách nhiệm:

- Tính toán tensor tốc độ cao
- Tối ưu gradient, tự động đạo hàm
- Chạy được trên CPU & GPU
- Hỗ trợ training ổn định và hiệu quả

- Keras

Keras là API cấp cao chạy trên TensorFlow, giúp xây dựng mô hình đơn giản hơn.

Trong đề tài, nhóm sử dụng các module:

- `keras.Sequential` để khai báo mô hình tuyến tính
- `layers.Conv2D` để xây dựng các tầng tích chập
- `layers.MaxPooling2D` để giảm kích thước
- `layers.Dropout` chống overfitting
- `layers.Dense` để tạo tầng Fully-Connected
- `keras.callbacks` để tối ưu quá trình huấn luyện
 - **EarlyStopping**
 - **ModelCheckpoint**
 - **ReduceLROnPlateau**

Các công cụ huấn luyện quan trọng được dùng trong code

- ✓ `model.compile(optimizer='adam')` → dùng Adam, tối ưu nhanh, ổn định
- ✓ `categorical_crossentropy` → phù hợp bài toán phân loại 10 lớp
- ✓ `ImageDataGenerator` → dùng để tạo **Data Augmentation** ngay khi train
- ✓ `model.fit()` → huấn luyện mô hình CNN theo batch
- ✓ `model.evaluate()` → đánh giá trên tập test

2.2.3. NumPy – Thư viện tính toán ma trận

- NumPy hỗ trợ:

- Xử lý dữ liệu ảnh dạng ma trận
 - Chuyển đổi kiểu dữ liệu (`astype('float32')`)
 - Tính toán vector hóa nhanh, rất quan trọng trong Deep Learning
- Trong code, NumPy được dùng cho:
- Chuẩn hóa pixel
 - Reshape dữ liệu
 - Trích xuất batch mẫu để test mô hình

2.2.4. Matplotlib – Tr trực quan hóa quá trình huấn luyện

- Matplotlib được sử dụng để:
- Vẽ biểu đồ **accuracy** và **loss** theo từng epoch
 - Lưu biểu đồ dưới dạng .png để đưa vào báo cáo
 - Hiển thị ảnh mẫu khi demo pipeline
- Điều này giúp quan sát:
- Mô hình có overfitting hay không
 - Mức độ hội tụ
 - Sự khác biệt giữa train và validation accuracy

2.2.5. OpenCV (cv2) – Xử lý ảnh

Thư viện OpenCV (nổi tiếng trong Computer Vision) được dùng trong:

- Đọc ảnh grayscale
- Resize ảnh (ví dụ cho shapes dataset)
- Tiền xử lý các ảnh đầu vào khi demo

OpenCV giúp thao tác với ảnh thuận tiện và nhanh chóng.

2.2.6. Callbacks – Các kỹ thuật tối ưu khi huấn luyện

Trong code, nhóm sử dụng 3 callback cực kỳ quan trọng:

✓ EarlyStopping

Dừng huấn luyện khi mô hình không còn cải thiện → tránh overfitting.

✓ ReduceLROnPlateau

Giảm learning rate khi mô hình đứng yên → giúp tối ưu tới điểm tốt nhất.

✓ ModelCheckpoint

Tự động lưu **mô hình tốt nhất** (best model), đảm bảo kết quả cuối cùng tối ưu.

2.2.7. Data Augmentation – Tăng cường dữ liệu

Đây là phần quan trọng nhất trong công nghệ xử lý dữ liệu MNIST của nhóm.

Nhóm sử dụng ImageDataGenerator để tạo ra:

- Xoay ảnh $\pm 15^\circ$
- Dịch trái/phải 15%
- Dịch lên/xuống 15%
- Zoom $\pm 15\%$
- Shear (biến dạng xiên)
- Fill với màu đen (0)

Điều này giúp mô hình:

- Khả năng tổng quát tốt hơn
- Nhận dạng được chữ số viết tay thật (không chỉ MNIST sạch)
- Tăng số lượng dữ liệu huấn luyện hiệu quả

PHẦN 3: TIỀN XỬ LÝ DỮ LIỆU

Tiền xử lý dữ liệu là bước nền tảng giúp mô hình CNN nhận dạng chính xác chữ số, chữ cái và hình học. Các ảnh viết tay thường có nhiễu, độ sáng không đều, kích thước khác nhau, ký tự không căn giữa, hoặc nền trắng/ngược sáng. Vì vậy, nhóm đã xây dựng một **pipeline gồm nhiều thuật toán xử lý ảnh**, mỗi thuật toán đảm nhiệm một vai trò riêng.

3.1. Chuyển đổi ảnh về thang xám (Grayscale Conversion)

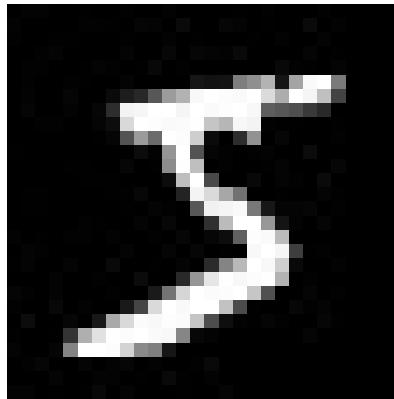
Mục đích

- Giảm số kênh màu từ RGB (3 kênh) → grayscale (1 kênh)
- Giảm độ phức tạp → giúp CNN học dễ hơn

Thuật toán

OpenCV dùng công thức chuẩn của ITU-R BT.601:

$$Gray = 0.299R + 0.587G + 0.114B$$



Ảnh sau chuyển xám

Tác dụng

- Làm nổi bật nét chữ/hình
- Loại bỏ thông tin màu không cần thiết

3.2. Chuẩn hóa pixel về khoảng [0, 1] (Normalization)

Công thức

$$I_{norm} = \frac{I}{255}$$

Lý do làm

- Đưa toàn bộ giá trị vào khoảng nhỏ giúp mô hình hội tụ nhanh hơn
- Tránh gradient quá lớn trong quá trình backpropagation

3.3. Tăng cường tương phản (Histogram Equalization / CLAHE)

3.3.1. Histogram Equalization (HE)

Mục đích

- Tăng độ tương phản bằng cách phân bố lại histogram của ảnh.

Ý tưởng thuật toán

- Tính histogram $H(i)$

- Tính hàm phân phối tích lũy (CDF):

$$CDF(i) = \sum_{j=0}^i H(j)$$

- Dùng CDF để ánh xạ lại pixel:

$$I_{new} = round\left(\frac{CDF(I) - CDF_{min}}{(M \times N) - CDF_{min}} \times 255\right)$$



Ảnh sau CLAHE

Tác dụng

- Chữ mờ → rõ hơn
- Hình nét đậm – nét nhạt → đồng đều

3.3.2. CLAHE (Contrast Limited Adaptive Histogram Equalization)

(dùng cho hình học – shapes)

Đặc điểm

HE toàn cục đôi khi làm ảnh bị “quá sáng”. CLAHE chia ảnh thành nhiều ô nhỏ (tiles) và equalize từng ô → tốt hơn với hình bị bóng tối hoặc ánh sáng mạnh.

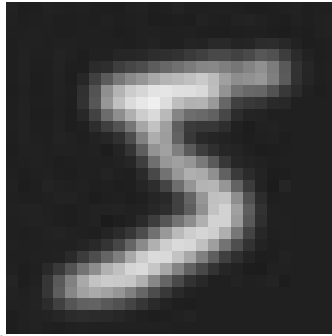
3.4. Khử nhiễu ảnh (Denoising)

3.4.1. Gaussian Blur

Công thức

Gaussian kernel 2D:

$$G(x, y) = \frac{1}{2\pi\sigma^2} e^{-\frac{x^2+y^2}{2\sigma^2}}$$



Ảnh sau áp dụng Gaussian Blur

Tác dụng

- Làm mượt ảnh, giảm nhiễu high-frequency
- Giúp thresholding ổn định hơn

3.4.2. Median Filter

- Thay pixel bằng **median** của lân cận
- Loại nhiễu muối tiêu (salt-and-pepper noise)

3.5. Phân ngưỡng (Thresholding)

3.5.1. Otsu's Threshold

Mục tiêu

Tìm ngưỡng T tối ưu để tách ảnh thành 2 lớp: foreground (ký tự) và background.

Thuật toán

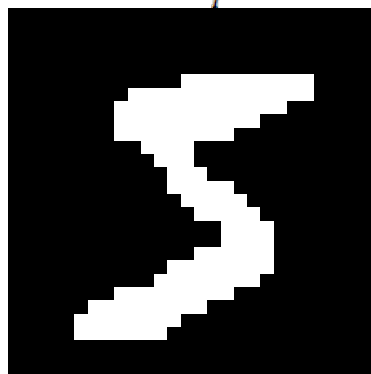
Otsu chọn T sao cho **độ lệch phương sai nội lớp (in-class variance) nhỏ nhất** hoặc **giữa 2 lớp lớn nhất**:

$$\sigma_b^2(T) = \omega_0(T)\omega_1(T)[\mu_0(T) - \mu_1(T)]^2$$

- w_0, w_1 : trọng số mỗi lớp
- μ_0, μ_1 : trung bình mỗi lớp

Ngưỡng tối ưu:

$$T^* = \arg \max_T \sigma_b^2(T)$$



Ảnh sau phân ngưỡng Otsu

Tác dụng

- Tách chữ trên nền cực kỳ hiệu quả
- Không cần chỉnh tay

3.5.2. Adaptive Threshold

Ý tưởng

- Tính ngưỡng cục bộ theo từng vùng nhỏ
- Phù hợp ảnh không đều sáng (glare, shadow)

3.6. Phép hình thái học (Morphological Operations)

Các phép hình thái học giúp chỉnh sửa hình dạng vùng trắng (ký tự/hình học).

Dùng kernel:

$$K = \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$$

3.6.1. Erosion (Co lại)

Công thức

Pixel = white nếu toàn bộ kernel fit vào pixel trắng

Tác dụng

- Xóa các điểm trắng nhỏ
- Làm nét mảnh hơn

3.6.2. Dilation (Giãn ra)

Ngược lại erosion.

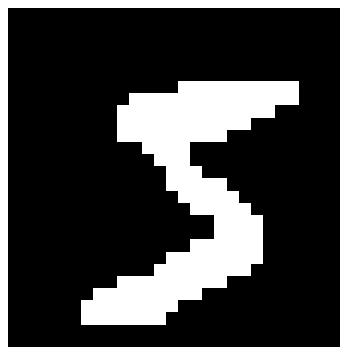
Tác dụng

- Làm nét dày lên
- Lấp điểm đen nhỏ trong chữ

3.6.3. Opening = Erosion → Dilation

Tác dụng

- Loại nhiễu trắng nhỏ
- Giữ lại cấu trúc chính của ký tự

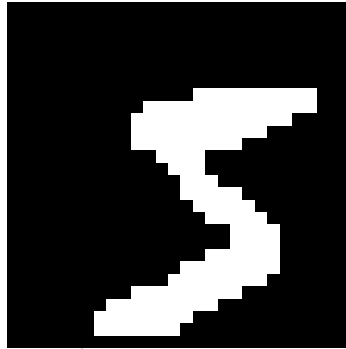


Ảnh sau khi opening

3.6.4. Closing = Dilation → Erosion

Tác dụng

- Lấp lỗ trong nét vẽ
- Làm ký tự hoàn chỉnh hơn



Ảnh sau khi closing

3.7. Phát hiện biên (Canny Edge Detection)

Bước 1: Làm mượt bằng Gaussian

Bước 2: Gradient bằng Sobel

$$G_x = I * S_x, \quad G_y = I * S_y$$

$$G = \sqrt{G_x^2 + G_y^2}$$

Bước 3: Non-maximum suppression

Loại bỏ điểm không phải biên.

Bước 4: Double threshold & Hysteresis

Tác dụng:

- Tách biên sắc nét → minh họa và debug pipeline

3.8. Connected Component Analysis

Phát hiện vùng liên thông của pixel trắng.

Ý tưởng

- Duyệt ảnh và đánh nhãn từng vùng pixel trắng nối liền nhau bằng:
4-connectivity hoặc *8-connectivity*

Tác dụng

- Tách ký tự chính khỏi nhiễu
- Minh họa ảnh rất tốt trong báo cáo

3.9. Convex Hull

Định nghĩa

Bao lồi của tập điểm S là tập nhỏ nhất bao trọn S.

Ví dụ dùng thuật toán Graham Scan / Andrew's monotone chain:

$$O(n \log n)$$

Tác dụng

- Tạo hình dạng bao ngoài mượt
- Dùng để phân tích hình học (đặc biệt shapes)

3.10. Tìm bounding box & cropping

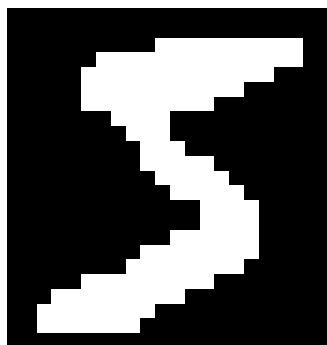
Thuật toán

- Dùng `cv2.findNonZero()` → lấy toàn bộ pixel trắng
- Tìm bounding box:

$$x, y, w, h = \text{boundingRect}(\text{coords})$$



Ảnh được tìm bounding



Cắt ảnh

Tác dụng

- Loại bỏ vùng nền thừa
- Tập trung mô hình vào ký tự

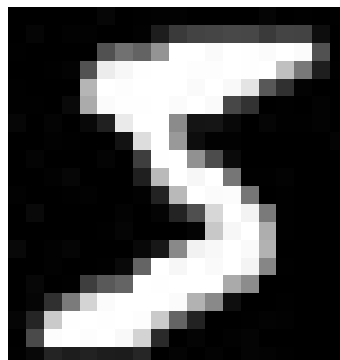
3.11. Resize giữ tỷ lệ (Aspect Ratio Preserving)

Công thức scale

$$\text{scale} = \min\left(\frac{\text{target_width}}{w}, \frac{\text{target_height}}{h}\right)$$

Resize:

$$w' = w \times \text{scale}, \quad h' = h \times \text{scale}$$



Ảnh được resize

Tác dụng

- Không làm méo ký tự
- Căn chỉnh thống nhất giữa các mẫu

3.12. Căn giữa hình vào canvas

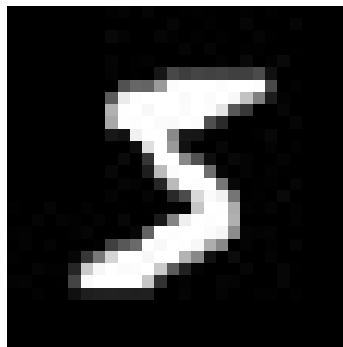
Thường dùng canvas:

- MNIST: 28×28
- Alphabet: 28×28
- Shapes: 64×64

Công thức đặt vào tâm

$$x_{offset} = \frac{W - w'}{2}$$

$$y_{offset} = \frac{H - h'}{2}$$

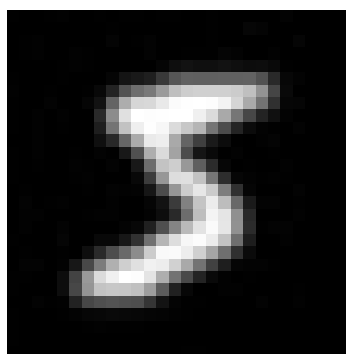


Ảnh được đặt vào giữa

3.13. Làm mượt cuối (Final Smoothing)

Dùng Gaussian Blur để:

- Làm nét chữ mềm hơn
- Giống phong cách MNIST gốc



Ảnh được làm mượt cuối

3.14. Chuẩn hóa đầu ra (Final Normalization)

$$I_{processed} = \frac{I}{255}$$

Reshape về dạng CNN yêu cầu:

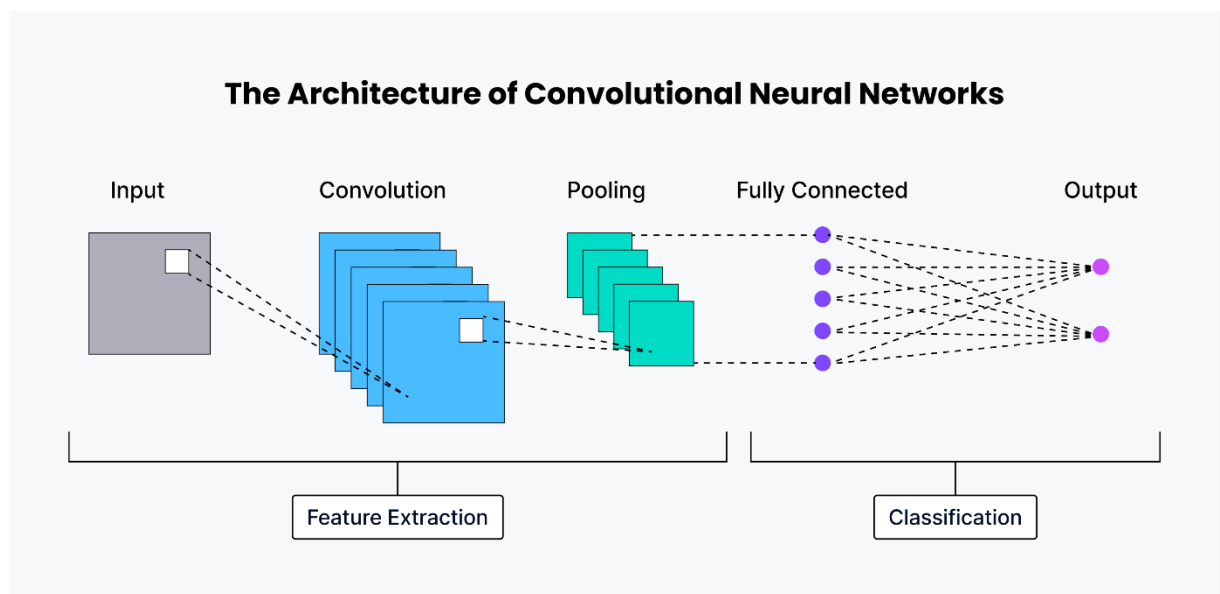
$$(1, H, W, 1)$$

PHẦN 4: KIẾN TRÚC MÔ HÌNH CNN

4.1. Tổng quan kiến trúc CNN

Mô hình **Convolutional Neural Network (CNN)** dùng trong đề tài có cấu trúc điển hình gồm 2 phần chính:

- **Khối trích xuất đặc trưng (Feature extractor)**
 - Các tầng: Convolution + ReLU + Pooling + Dropout
 - Nhiệm vụ: tự động học ra các đặc trưng về hình dạng (cạnh, nét cong, góc, độ dày nét viết,...).
- **Khối phân loại (Classifier)**
 - Các tầng: Flatten + Dense (Fully-connected) + Softmax
 - Nhiệm vụ: từ đặc trưng đã trích xuất → quyết định đây là chữ số nào (0–9).



Cấu trúc mô hình CNN

Kiến trúc này được cài đặt trong file `Train_MNIST.ipynb`.

4.2. Kiến trúc chi tiết mô hình CNN cho MNIST

Trong code, mô hình cho MNIST được định nghĩa như sau:

```

model = keras.Sequential([
    # Input layer
    layers.Input(shape=(28, 28, 1)),

    # Convolutional Block 1
    layers.Conv2D(32, kernel_size=(3, 3), activation='relu', padding='same'),
    layers.MaxPooling2D(pool_size=(2, 2)),
    layers.Dropout(0.25),

    # Convolutional Block 2
    layers.Conv2D(64, kernel_size=(3, 3), activation='relu', padding='same'),
    layers.MaxPooling2D(pool_size=(2, 2)),
    layers.Dropout(0.25),

    # Convolutional Block 3
    layers.Conv2D(64, kernel_size=(3, 3), activation='relu', padding='same'),
    layers.Dropout(0.25),

    # Fully Connected layers
    layers.Flatten(),
    layers.Dense(128, activation='relu'),
    layers.Dropout(0.5),
    layers.Dense(10, activation='softmax')
])

```

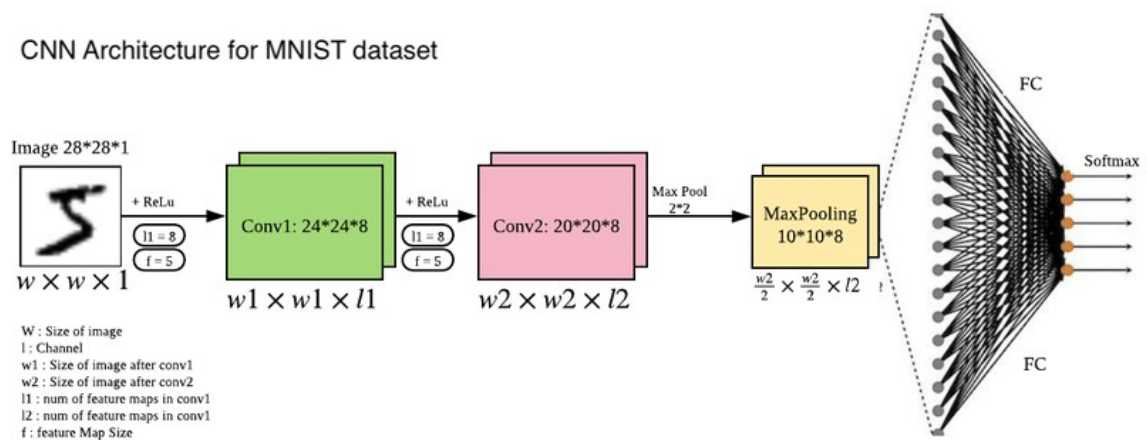
Ta có thể mô tả lại thành bảng:

Thứ tự	Tầng	Kích thước output	Vai trò chính
1	Input	28×28×1	Ảnh số xám 28×28
2	Conv2D(32, 3×3, ReLU)	28×28×32	Học cạnh/nét cơ bản
3	MaxPooling(2×2)	14×14×32	Giảm kích thước, giữ đặc trưng chính
4	Dropout 0.25	14×14×32	Giảm overfitting
5	Conv2D(64, 3×3, ReLU)	14×14×64	Học đặc trưng phức tạp hơn
6	MaxPooling(2×2)	7×7×64	Tiếp tục giảm kích thước
7	Dropout 0.25	7×7×64	Giảm overfitting
8	Conv2D(64, 3×3, ReLU)	7×7×64	Tinh chỉnh đặc trưng cấp cao
9	Dropout 0.25	7×7×64	Giảm overfitting

10	Flatten	3136	“Trải phẳng” feature map
11	Dense(128, ReLU)	128	Học tổ hợp đặc trưng để phân loại
12	Dropout 0.5	128	Regularization mạnh
13	Dense(10, Softmax)	10	Xác suất 10 lớp (digit 0–9)

4.3. Giải thích từng nhóm tầng và ý nghĩa

CNN Architecture for MNIST dataset



Mô hình CNN với tập dữ liệu MNIST

4.3.1. Tầng tích chập (Convolutional Layer – Conv2D)

Công thức tích chập 2D (đơn giản hóa):

Giả sử đầu vào là x , kernel (filter) là w , đầu ra là y :

$$y_{i,j,k} = \sum_{u,v,c} x_{i+u,j+v,c} \cdot w_{u,v,c,k} + b_k$$

- k : chỉ số filter (tạo ra 1 “feature map”)
- (u,v) : tọa độ trong kernel
- c : kênh màu (ở đây chỉ 1 vì ảnh grayscale)

Trong mô hình:

- **Conv1**: 32 filters 3×3 , padding='same'
 → từ $28 \times 28 \times 1 \rightarrow 28 \times 28 \times 32$
 → học được cạnh, nét cong đơn giản, điểm sáng/tối.
- **Conv2**: 64 filters 3×3
 → $14 \times 14 \times 32 \rightarrow 14 \times 14 \times 64$
 → học pattern phức tạp hơn (góc, hình dạng số).
- **Conv3**: 64 filters 3×3
 → $7 \times 7 \times 64 \rightarrow 7 \times 7 \times 64$ (giữ kích thước, tăng “chiều sâu đặc trưng”).

Tác dụng:

CNN **tự học feature** thay vì phải tự thiết kế thủ công đặc trưng (HOG, SIFT,...).

4.3.2. Hàm kích hoạt ReLU

Sau mỗi Conv2D, mô hình dùng:

`activation='relu'`

Công thức ReLU:

$$ReLU(x) = \max(0, x)$$

Tác dụng:

- Tạo tính phi tuyến cho mạng (nếu không thì chỉ là phép biến đổi tuyến tính).
- Làm mô hình học được quan hệ phức tạp.
- Tính toán đơn giản → train nhanh, hạn chế vanishing gradient.

4.3.3. Tầng Pooling (MaxPooling2D)

Ví dụ:

`layers.MaxPooling2D(pool_size=(2, 2))`

Ý tưởng:

- Chia feature map thành các khối 2×2 , lấy **giá trị lớn nhất** trong mỗi khối.

Tác dụng:

- Giảm kích thước từ:
 - $28 \times 28 \rightarrow 14 \times 14$
 - $14 \times 14 \rightarrow 7 \times 7$
- Giảm số lượng tham số, giảm chi phí tính toán.
- Giúp mô hình **bớt nhạy với dịch chuyển nhỏ** (translation invariance).

Trong thực tế: nếu nét chữ hơi lệch 1–2 pixel thì kết quả sau pooling không thay đổi nhiều.

4.3.4. Dropout – Chống overfitting

Trong code:

`layers.Dropout(0.25)`

`layers.Dropout(0.5)`

Ý tưởng:

- Trong quá trình huấn luyện, mỗi bước sẽ “**tắt**” **ngẫu nhiên** một tỉ lệ neuron (25% hoặc 50%).

Tác dụng:

- Buộc mạng **không phụ thuộc quá nhiều** vào một vài neuron.
- Giúp mô hình generalize tốt hơn trên dữ liệu mới.

Dropout 0.5 ở tầng Dense 128 là khá mạnh → giúp giảm overfitting cho bài toán MNIST vốn dễ.

4.3.5. Flatten + Dense – Khối phân loại

Sau khi qua 3 khối Conv + Pooling, ta có feature map kích thước:

$$7 \times 7 \times 64 = \mathbf{3136} \text{ giá trị.}$$

Tầng **Flatten** biến tensor $7 \times 7 \times 64$ thành vector 3136 chiều.
Sau đó:

Dense(128, activation='relu')
Dense(10, activation='softmax')

- **Dense 128:** học tổ hợp đặc trưng cấp cao (kiểu: “có vòng tròn + nét xiên” → có thể là số 9, v.v.).
- **Dense 10 + Softmax:** cho ra xác suất 10 lớp.

Công thức Softmax cho lớp k:

$$p_k = \frac{e^{z_k}}{\sum_{j=1}^{10} e^{z_j}}$$

z_k : logit (đầu ra chưa chuẩn hoá) của lớp k
 p_k : xác suất số k

Mô hình sẽ chọn:

$$\hat{y} = \underset{k}{\operatorname{arg\,max}} p_k$$

làm chữ số dự đoán.

4.4. Mối liên hệ giữa kiến trúc CNN và dữ liệu MNIST

- Kích thước input $28 \times 28 \times 1$ được thiết kế **khớp với MNIST**.
- 3 khối Conv giúp mô hình **từ từ học lên các đặc trưng cấp cao**:
 - Conv1: nét, cạnh, chấm...
 - Conv2: đoạn cong, một nửa chữ số
 - Conv3: pattern gần giống toàn bộ chữ số.
- Pooling giúp mô hình **chịu được lệch vị trí, xoay nhẹ**.
- Dense 128 và Softmax 10 tương ứng với **bài toán 10 lớp (0–9)**.

Kiến trúc này cân bằng giữa:

- Độ sâu vừa đủ để đạt độ chính xác cao
- Số tham số không quá lớn nên train nhanh, phù hợp môi trường học phần.

PHẦN 5: KẾT QUẢ HUẤN LUYỆN MÔ HÌNH

5.1. Nhận dạng chữ số MNIST

5.1.1. Thiết lập huấn luyện

Cấu hình huấn luyện chính:

- **Dữ liệu:**
 - Train: 60.000 ảnh MNIST (28×28, grayscale)
 - Test: 10.000 ảnh MNIST
- **Tiền xử lý:**
 - Chuẩn hoá pixel về [0, 1]
 - Reshape về dạng (28, 28, 1)
 - One-hot encoding nhãn về dạng vector 10 chiều
- **Data Augmentation:**
 - rotation_range=15 (xoay $\pm 15^\circ$)
 - width_shift_range=0.15 (dịch ngang 15%)
 - height_shift_range=0.15 (dịch dọc 15%)
 - zoom_range=0.15 (phóng to/thu nhỏ 15%)
 - shear_range=0.1 (biến dạng xiên nhẹ)
 - fill_mode='constant', cval=0 (vùng trống được đổ màu đen)

Điều này giúp mô hình “làm quen” với những kiểu chữ số:

- bị lệch
- hơi méo
- hơi xoay
- viết không đúng chính giữa

nên khi áp dụng với ảnh chữ viết tay thật (vẽ bằng chuột / bút / ảnh chụp) sẽ khoan dung hơn.

Thông số huấn luyện:

- **Batch size:** 128
- **Số epoch tối đa:** 30
- **Optimizer:** Adam
- **Loss:** categorical_crossentropy
- **Metric:** accuracy
- **Callbacks:**

```

Model đã sẵn sàng!

# Thiết lập callbacks
callbacks = [
    keras.callbacks.EarlyStopping(
        monitor='val_loss',
        patience=5,
        restore_best_weights=True,
        verbose=1
    ),
    keras.callbacks.ReduceLROnPlateau(
        monitor='val_loss',
        factor=0.5,
        patience=3,
        verbose=1
    )
]

print("✅ Callbacks configured!")

✅ Callbacks configured!

```

Nhờ **EarlyStopping**, mô hình sẽ dừng sớm khi validation loss không còn cải thiện, tránh train quá lâu và overfitting.

5.1.2. Kết quả định lượng trên tập kiểm tra

Sau khi huấn luyện với Data Augmentation, notebook ghi lại kết quả đánh giá trên test set như sau:

```

Epoch 1/30
/usr/local/lib/python3.12/dist-packages/keras/src/trainers/data_adapters/py_dataset_adapter.py:121: UserWarning: Your `PyDataset` class should call `self._warn_if_super_not_called()`
  self._warn_if_super_not_called()
468/468 ————— 32s 53ms/step - accuracy: 0.5918 - loss: 1.1760 - val_accuracy: 0.9779 - val_loss: 0.0691 - learning_rate: 0.0010
Epoch 2/30
1/468 ————— 3s 8ms/step - accuracy: 0.9453 - loss: 0.2531
/usr/local/lib/python3.12/dist-packages/keras/src/trainers/epoch_iterator.py:116: UserWarning: Your input ran out of data; interrupting training. Make
  self._interrupted_warning()
=====
468/468 ————— 1s 1ms/step - accuracy: 0.9453 - loss: 0.2531 - val_accuracy: 0.9780 - val_loss: 0.0691 - learning_rate: 0.0010
Epoch 3/30
468/468 ————— 30s 42ms/step - accuracy: 0.9191 - loss: 0.2695 - val_accuracy: 0.9840 - val_loss: 0.0471 - learning_rate: 0.0010
Epoch 4/30
468/468 ————— 1s 1ms/step - accuracy: 0.8984 - loss: 0.2988 - val_accuracy: 0.9846 - val_loss: 0.0457 - learning_rate: 0.0010
Epoch 5/30
468/468 ————— 19s 41ms/step - accuracy: 0.9438 - loss: 0.1935 - val_accuracy: 0.9871 - val_loss: 0.0398 - learning_rate: 0.0010
Epoch 6/30
468/468 ————— 1s 2ms/step - accuracy: 0.9766 - loss: 0.1155 - val_accuracy: 0.9880 - val_loss: 0.0368 - learning_rate: 0.0010
Epoch 7/30
468/468 ————— 19s 40ms/step - accuracy: 0.9524 - loss: 0.1616 - val_accuracy: 0.9912 - val_loss: 0.0271 - learning_rate: 0.0010
Epoch 8/30
468/468 ————— 1s 1ms/step - accuracy: 0.9453 - loss: 0.2351 - val_accuracy: 0.9910 - val_loss: 0.0282 - learning_rate: 0.0010
Epoch 9/30
468/468 ————— 19s 41ms/step - accuracy: 0.9586 - loss: 0.1416 - val_accuracy: 0.9923 - val_loss: 0.0235 - learning_rate: 0.0010
Epoch 10/30
468/468 ————— 1s 1ms/step - accuracy: 0.9766 - loss: 0.0738 - val_accuracy: 0.9916 - val_loss: 0.0241 - learning_rate: 0.0010
Epoch 11/30
468/468 ————— 19s 40ms/step - accuracy: 0.9621 - loss: 0.1265 - val_accuracy: 0.9921 - val_loss: 0.0225 - learning_rate: 0.0010
Epoch 12/30
468/468 ————— 1s 2ms/step - accuracy: 0.9766 - loss: 0.0956 - val_accuracy: 0.9918 - val_loss: 0.0232 - learning_rate: 0.0010
Epoch 13/30
468/468 ————— 20s 42ms/step - accuracy: 0.9665 - loss: 0.1151 - val_accuracy: 0.9928 - val_loss: 0.0217 - learning_rate: 0.0010
Epoch 14/30
...

=====
✅ TRAINING HOÀN THÀNH!

```

```
Đánh giá model trên test set...

=====
KẾT QUẢ CUỐI CÙNG
=====
Test Accuracy: 0.9948 (99.48%)
Test Loss: 0.0138
=====
```

- **Test Accuracy: 0.9948 $\approx$ 99,48%**
- **Test Loss: 0.0138**

Nghĩa là:

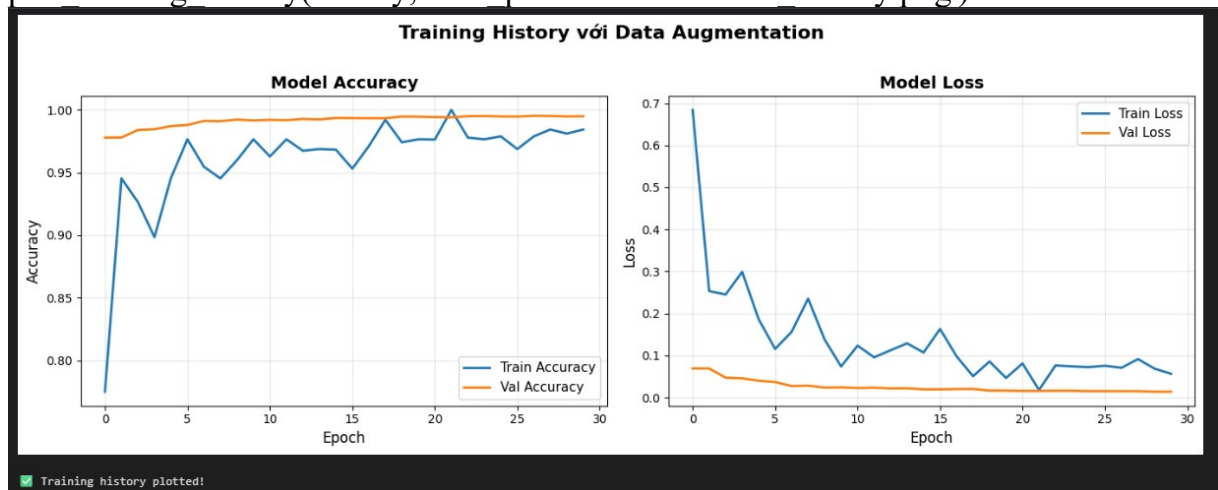
Mô hình dự đoán đúng khoảng **99,48%** số ảnh chữ số viết tay trên tập test MNIST. Đây là mức độ chính xác **rất cao** đối với một mô hình CNN tương đối đơn giản, chứng tỏ:

- Kiến trúc CNN được thiết kế hợp lý
- Data Augmentation + tiền xử lý hoạt động hiệu quả
- Mô hình học được các đặc trưng ổn định của chữ số, không chỉ “học thuộc” dữ liệu train.

5.1.3. Đồ thị quá trình huấn luyện (Loss & Accuracy)

Trong notebook, nhóm có vẽ và lưu biểu đồ quá trình huấn luyện bằng hàm:

`plot_training_history(history, save_path='models/mnist_history.png')`

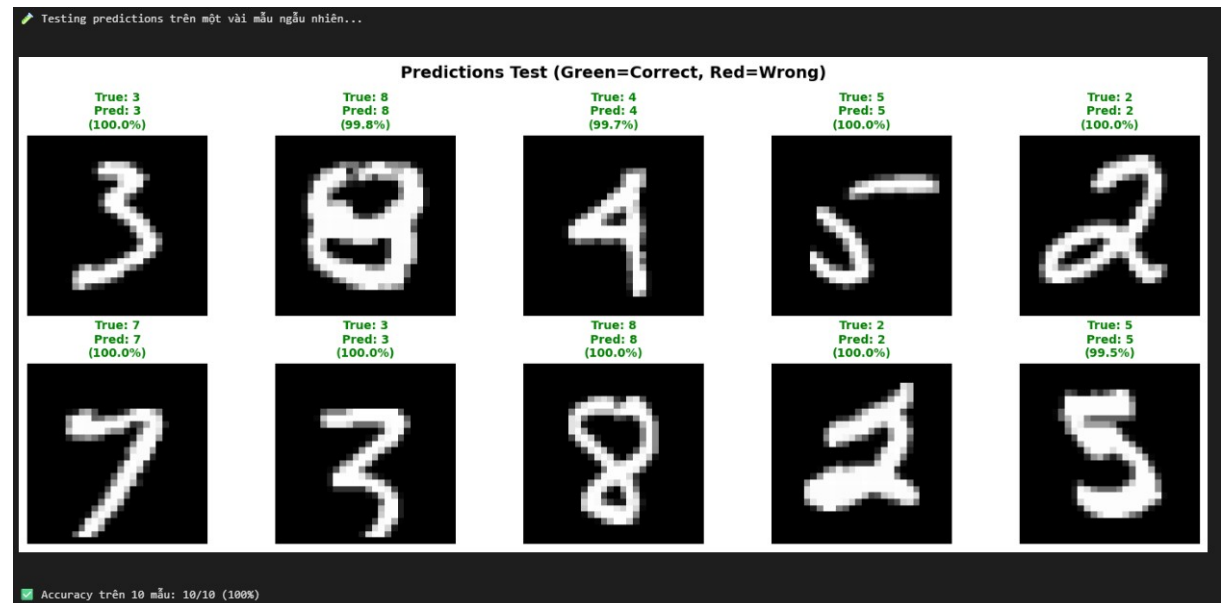


Biểu đồ gồm 2 phần:

- **Training vs Validation Accuracy theo Epoch**
 - Đường **train_accuracy** tăng nhanh và tiến tới gần 1.0 (100%) sau một số epoch.
 - Đường **val_accuracy** (accuracy trên test/validation) cũng tăng ổn định và tiệm cận khoảng **0.99+**.
 - Hai đường không bị tách xa nhau $\rightarrow$ dấu hiệu tốt, mô hình **không bị overfitting mạnh**.
- **Training vs Validation Loss theo Epoch**

- **train_loss** giảm nhanh lúc đầu rồi dần dần nhỏ lại và ổn định.
- **val_loss** cũng giảm theo, sau đó dao động nhẹ quanh một giá trị thấp ($\sim 0.01-0.02$).
- Khi **val_loss** không còn cải thiện, **EarlyStopping** kích hoạt và dừng huấn luyện.

Test dự đoán 1 vài mẫu ảnh:



=> Độ chính xác 100%

5.1.4. Nhận xét & đánh giá

Từ các kết quả trên, có thể rút ra một số nhận xét:

- **Mô hình CNN đã học rất tốt đặc trưng của chữ số MNIST**
 - Accuracy gần 99,5% là mức rất cao so với nhiều mô hình cơ bản.
 - Loss nhỏ cho thấy mô hình tin tưởng khá cao vào các dự đoán đúng.
- **Data Augmentation đóng vai trò quan trọng**
 - Giúp mô hình học từ các biến dạng thường thấy trong chữ viết tay thực tế: xoay, dịch, zoom, méo.
 - Làm cho mô hình bền vững hơn khi áp dụng vào ảnh không “sạch” như MNIST.
- **Các kỹ thuật huấn luyện nâng cao mang lại hiệu quả rõ rệt**
 - **EarlyStopping + ReduceLROnPlateau** → giúp hội tụ nhanh, tránh lãng phí epoch, tránh overfitting.
 - **ModelCheckpoint** → luôn lưu lại mô hình tốt nhất theo **val_accuracy**, đảm bảo khi deploy dùng phiên bản tối ưu.
- **Tiền xử lý dữ liệu + chuẩn hoá đầu vào rất quan trọng**
 - Việc chuẩn hoá ảnh, căn giữa, scale về $[0, 1]$ giúp mô hình nhìn dữ liệu một cách nhất quán.

- Nếu không có pipeline tiền xử lý tốt, accuracy khi áp dụng vào chữ viết tay ngoài đời có thể giảm đáng kể.

5.2. Nhận dạng hình học

5.2.1. Tổng quan mô hình & dữ liệu

- **Dataset:** shapes_dataset_v3
- **Số lượng lớp:** 10 classes
- ['arrow', 'circle', 'cross', 'hexagon', 'octagon', 'pentagon', 'rectangle', 'rhombus', 'star', 'triangle']
- **Số mẫu:**
 - Train: 16,000 ảnh
 - Test: 4,000 ảnh
- **Kích thước ảnh:** 64×64 – grayscale

```
Drive already mounted at /content/drive; to attempt to forcibly remount, call drive.mount("/content/drive", force_remount=True).
Đang giải nén /content/drive/MyDrive/shapes_dataset_v2.zip vào /content/datasets...
Giải nén hoàn tất!
✅ Đã tìm thấy dataset tại: /content/datasets/shapes_dataset_v3
Số lượng classes: 10
Classes: ['arrow', 'circle', 'cross', 'hexagon', 'octagon', 'pentagon', 'rectangle', 'rhombus', 'star', 'triangle']

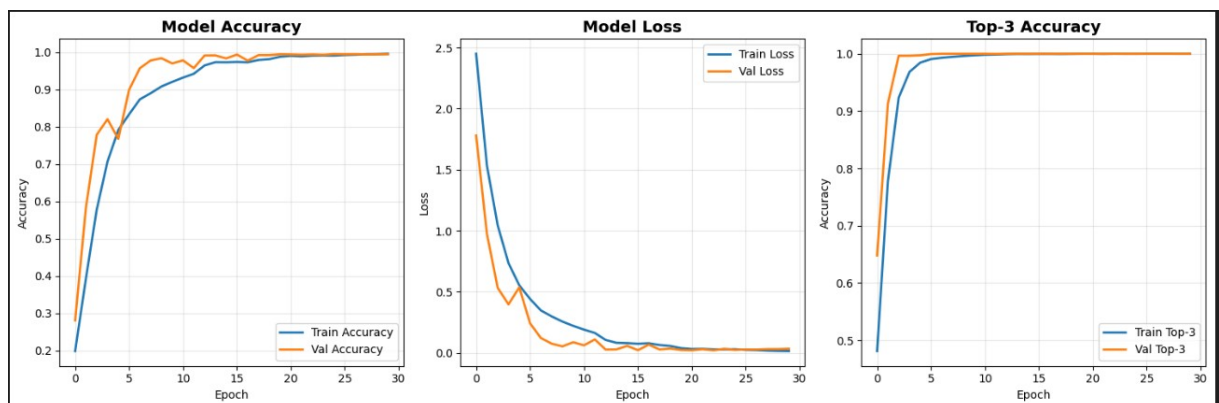
Đang tải tập huấn luyện (train)...
Found 16000 files belonging to 10 classes.
Đang tải tập kiểm tra (test)...
Found 4000 files belonging to 10 classes.
Đã tìm thấy 10 lớp: ['arrow', 'circle', 'cross', 'hexagon', 'octagon', 'pentagon', 'rectangle', 'rhombus', 'star', 'triangle']

Model: "sequential_1"
```

Tải dữ liệu huấn luyện

- **Tiền xử lý:**
 - Rescaling 1/255
 - Data Augmentation mạnh: rotation, translation, zoom, contrast
- **Kiến trúc mô hình:** CNN 3 block + BatchNorm + Dropout
 - Tổng số tham số: **2,421,354**
 - Dense 256 → 128 → Softmax(10)

5.2.2. Biểu đồ quá trình huấn luyện



Biểu đồ kết quả

✓ **Model Accuracy**

- Train accuracy tăng đều từ **15%** → **99.5%**
- Val accuracy tăng mạnh và ổn định từ **28%** → **99.4%**
- Không có hiện tượng overfitting nặng, đường train–val khá sát nhau.

✓ **Model Loss**

- Train loss giảm từ **~2.8** → **~0.01**
- Val loss giảm từ **1.78** → **~0.02**
- Loss hội tụ rất tốt, không dao động mạnh.

✓ **Top-3 Accuracy**

- Tăng nhanh từ 40% → **100%**
- Val top-3 accuracy đạt **100%** rất sớm (~epoch 6)
→ Điều này phù hợp vì chỉ có 10 classes, không khó để top-3 đạt hoàn hảo.

5.2.3. Bảng kết quả huấn luyện

Chỉ số	Giá trị
Train Accuracy	99.55%
Validation Accuracy	99.45%
Top-3 Accuracy	100%
Test Accuracy	99.35%
Test Loss	0.0214

Kết luận: **mô hình hoạt động cực kỳ tốt**, gần mức hoàn hảo.

5.2.4. Phân tích chi tiết kết quả

(1) **Mô hình học tốt và nhanh**

Ở epoch 3–4, val accuracy đã vượt **80%**, chứng tỏ:

- Kiến trúc CNN phù hợp
- Augmentation mạnh giúp mô hình học được biến dạng hình học (xoay, zoom, méo)

(2) **Accuracy rất cao (99.3–99.4%)**

Điều này nhờ:

- Dataset cân bằng, rõ ràng, ít nhiễu
- 3 block Convolution + BatchNorm → ổn định gradient
- Dropout 0.25–0.5 trong từng block giúp tránh overfitting
- Dense layers 256 + 128 giúp mô hình học pattern phức tạp

(3) **Không bị overfitting**

- Train acc $\approx$ val acc
- Train loss $\approx$ val loss
→ Rất khác với mô hình MNIST/Alphabet, vì dataset shapes **dễ hơn**, ít nhiễu hơn.

(4) Learning Rate Schedule hiệu quả

- ReduceLROnPlateau tự động giảm LR khi val_loss không cải thiện
- Learning rate giảm theo schedule:
0.001 $\rightarrow$ 0.0005 $\rightarrow$ 0.00025 $\rightarrow$ 0.000125 $\rightarrow$ 0.0000625
→ giúp mô hình mài sát nghiệm tối ưu, đạt 99.4%

(5) Cấu trúc hình học đơn giản $\rightarrow$ mô hình dễ phân biệt

Ví dụ:

- circle vs hexagon vs octagon có đường biên khác nhau
- triangle, star, cross có pattern rất đặc trưng
- Data Augmentation xoay/zoom giúp mô hình học các phiên bản biến dạng

5.2.5. Đánh giá độ tin cậy của mô hình

Mô hình hoạt động tốt trong các điều kiện:

- ✓ ảnh rõ nét, có độ tương phản rõ
- ✓ hình nằm gần giữa ảnh
- ✓ không chồng chéo nhiều thành phần
- ✓ kích thước hình không quá nhỏ

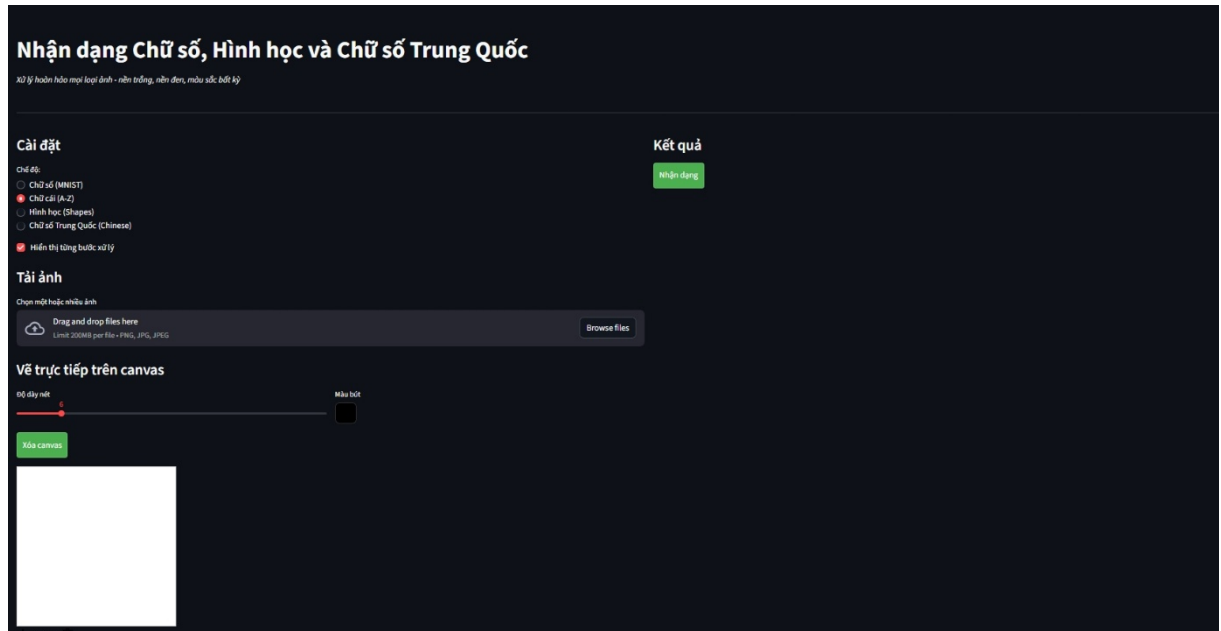
Một số trường hợp có thể gây lỗi:

- ✗ hình quá méo hoặc quá nhỏ
- ✗ nền xám nhạt, không đủ tương phản
- ✗ hình bị vỡ nét hoặc nhiễu mạnh
- ✗ hình bị cong/tròn kiểu handwritten (đối với shapes tự vẽ tay)

PHẦN 6: KẾT QUẢ DEMO VÀ HƯỚNG PHÁT TRIỂN

6.1. Demo nhận dạng ký tự

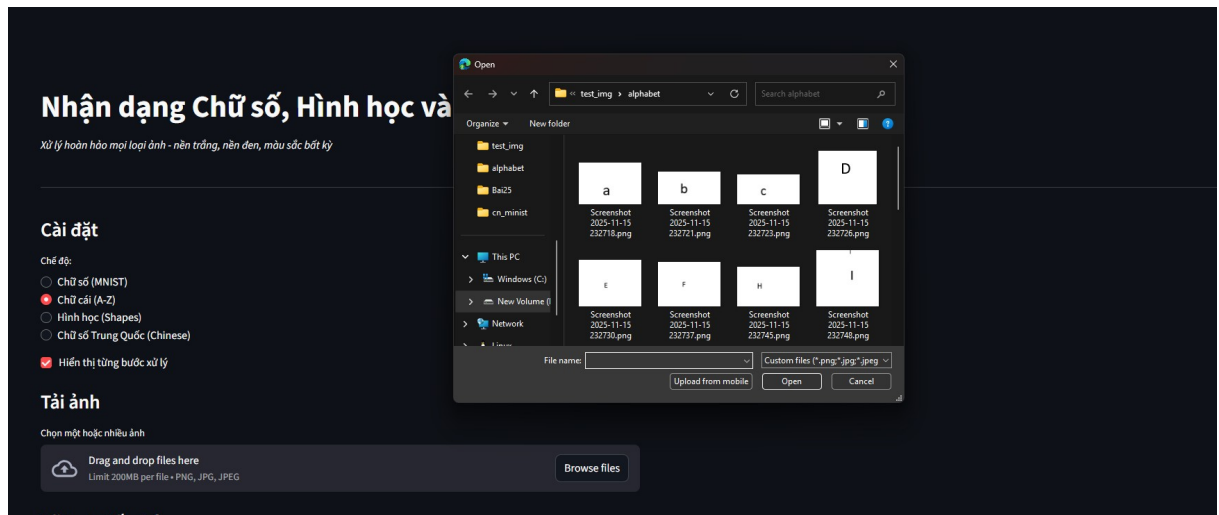
1. Giao diện của ứng dụng



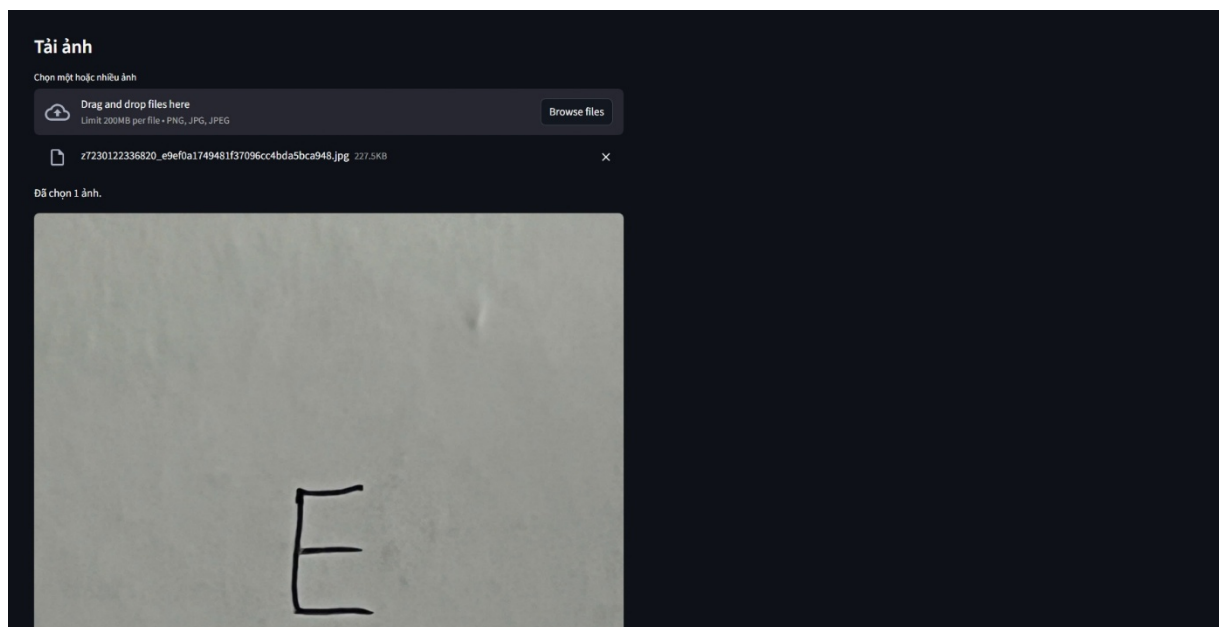
2. Chọn chế độ nhận diện



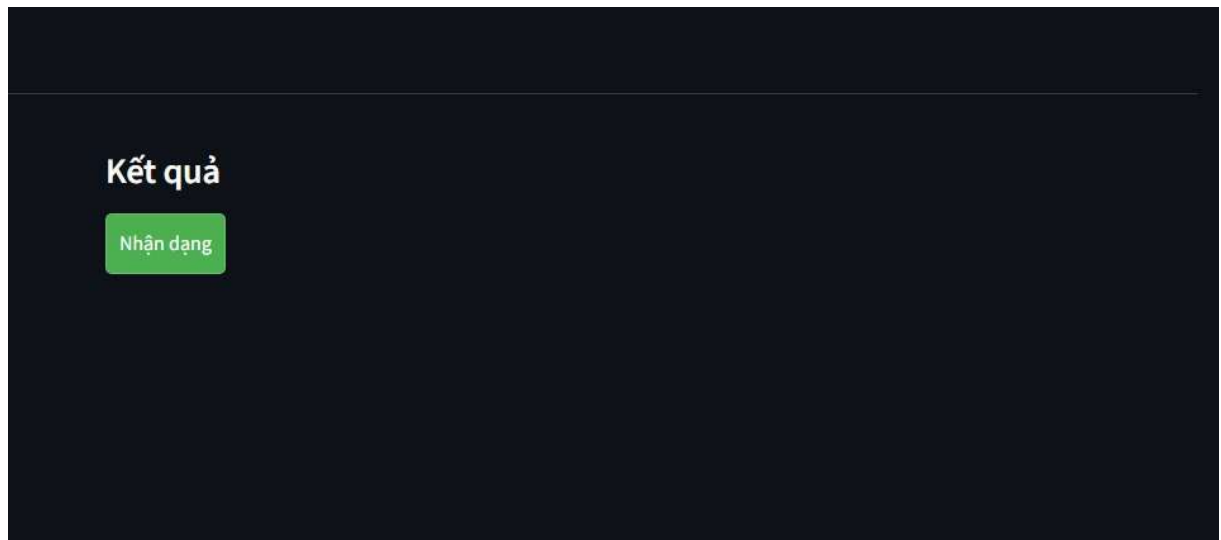
3. Chọn ảnh để upload



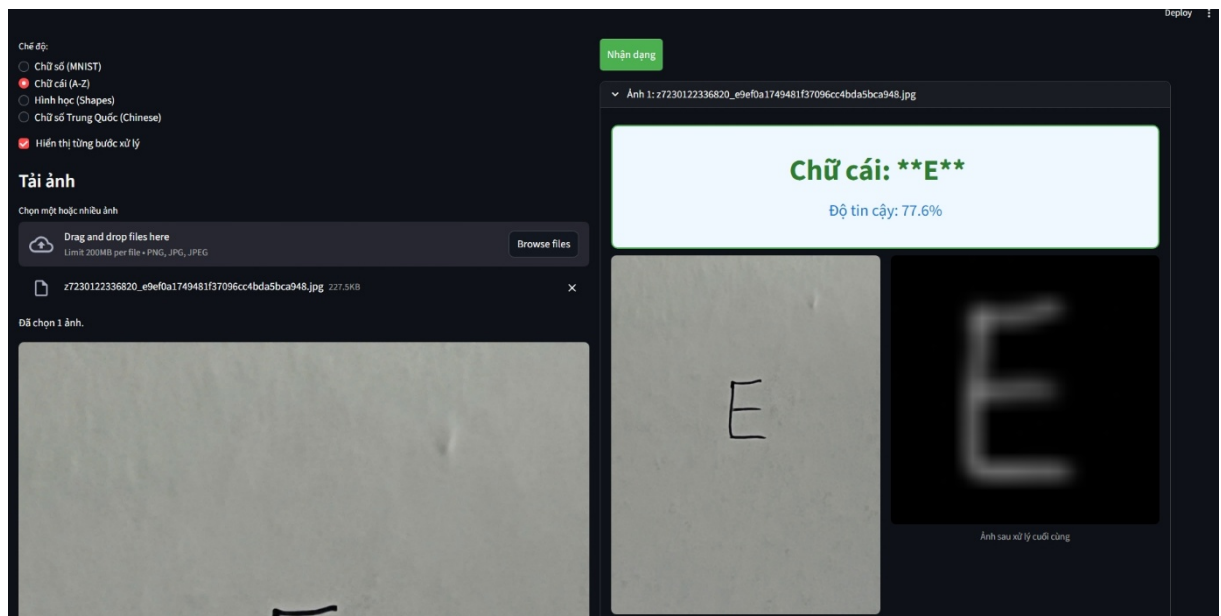
4. Ảnh hiển thị trên giao diện



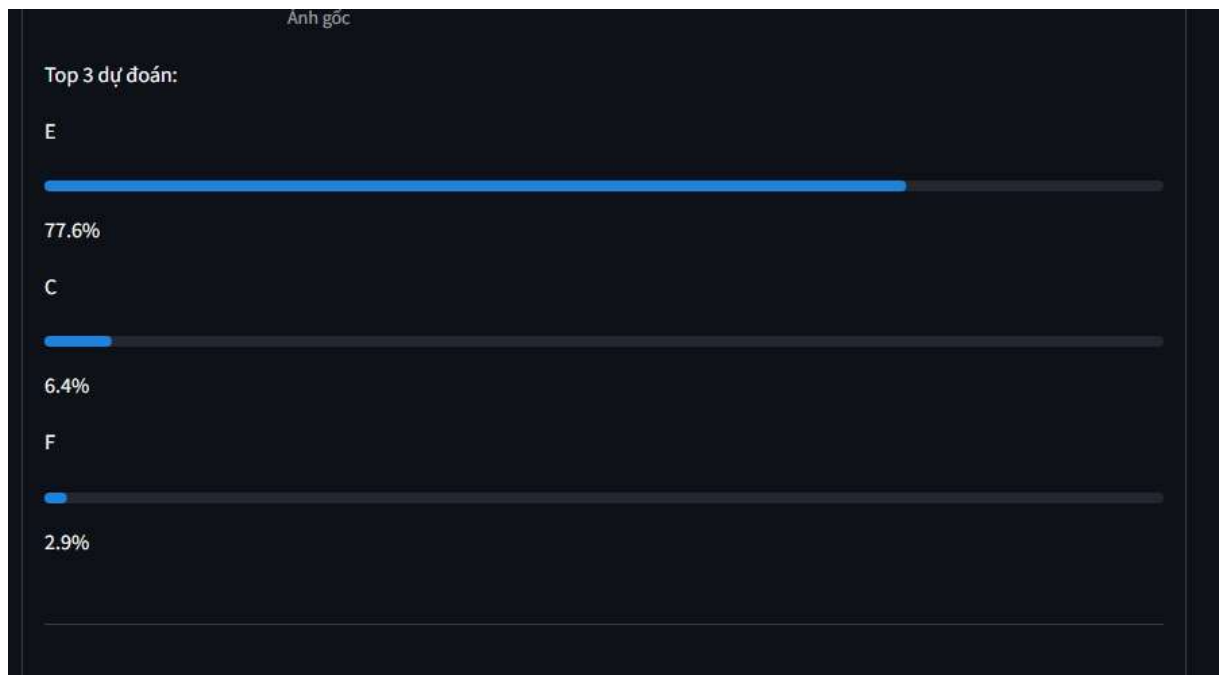
5. Chọn chức năng nhận dạng



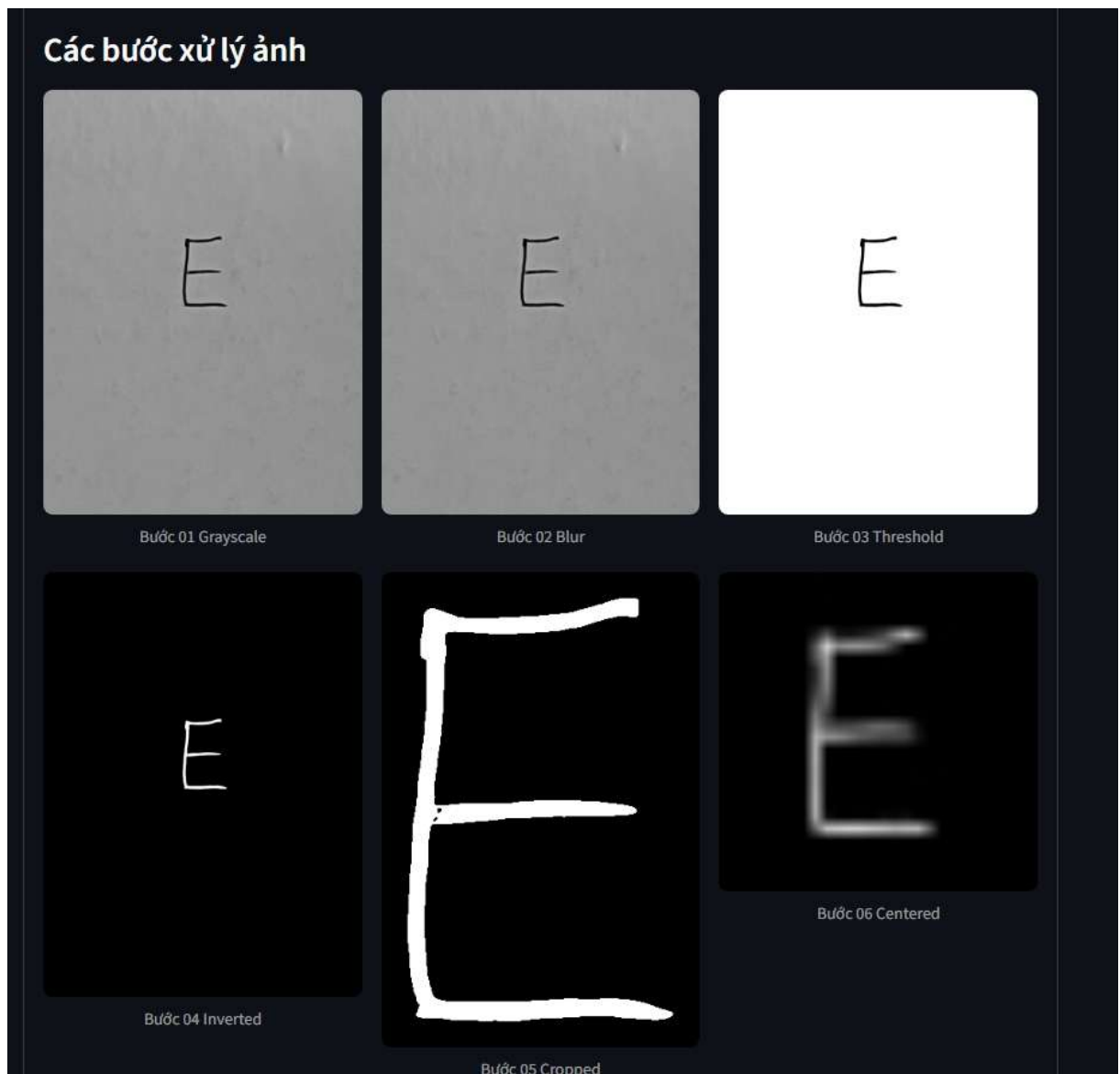
6. Hiển thị kết quả dự đoán



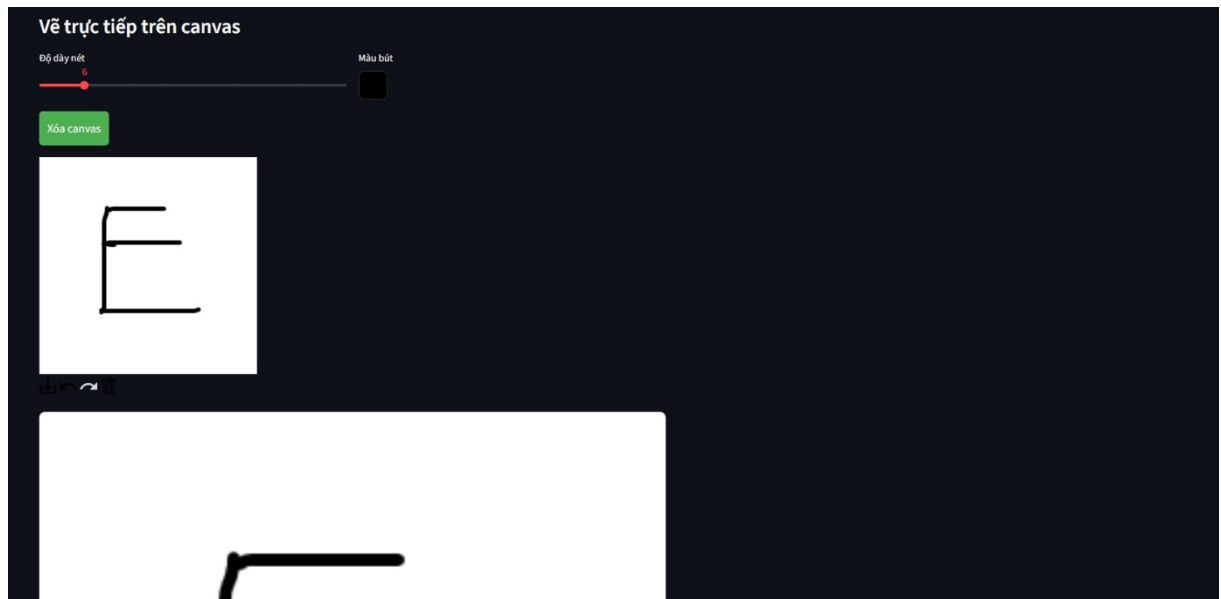
7. Danh sách top các dự đoán



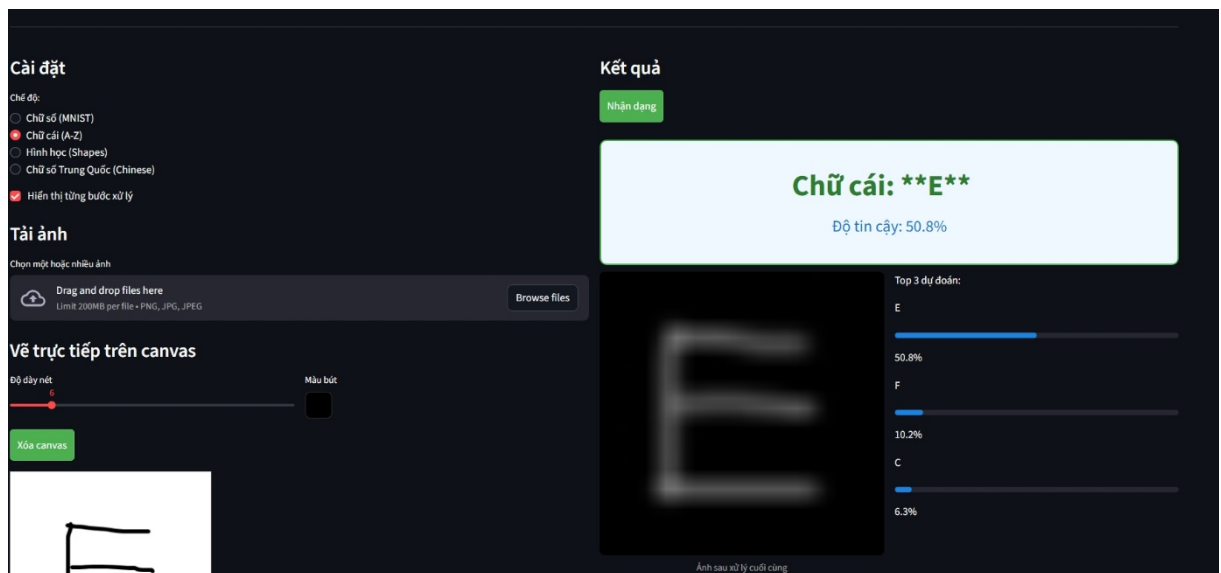
8. Hiển thị các bước xử lý



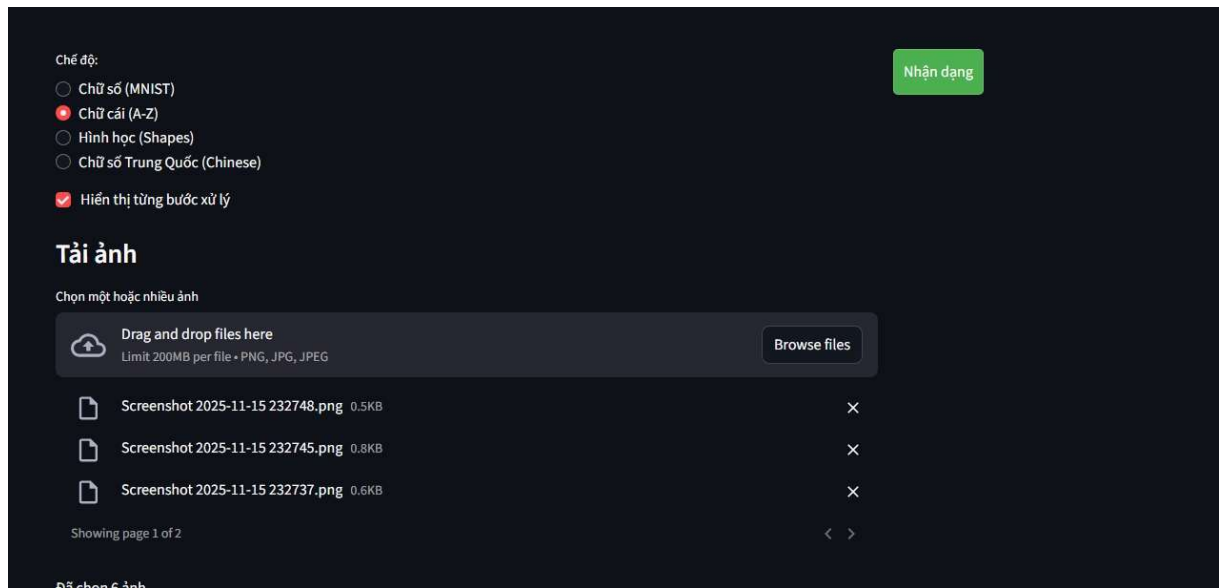
9. Tính năng vẽ trực tiếp



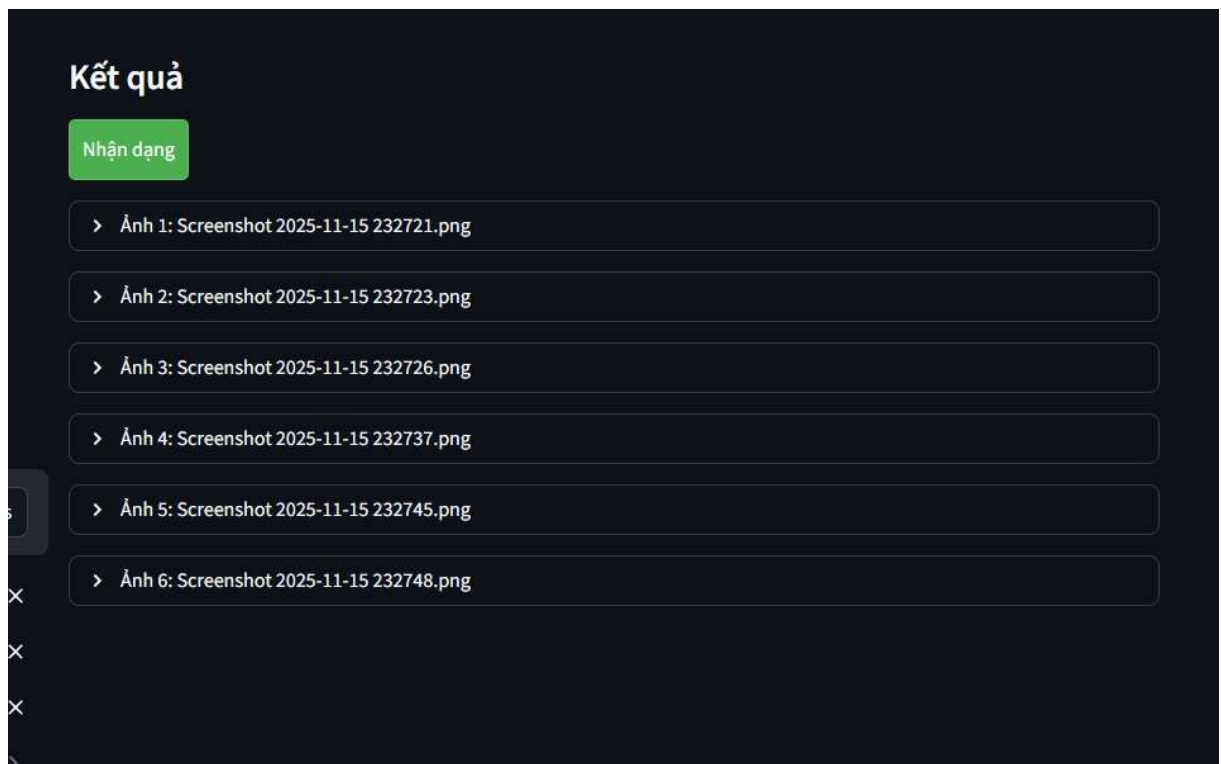
10. Kết quả dự đoán



11. Upload nhiều ảnh 1 lúc



12. Kết quả theo danh sách



6.2. Hướng phát triển

Trong tương lai, hệ thống có thể được mở rộng theo nhiều hướng để tăng tính ứng dụng và hiệu quả. Trước hết, nhóm có thể hợp nhất ba mô hình riêng biệt (chữ số, chữ cái và hình học) thành một mô hình duy nhất đa nhiệm, giúp giảm chi phí lưu trữ và tối ưu hóa tốc độ suy luận. Bên cạnh đó, việc xây dựng một phiên bản web hoặc ứng dụng di động sẽ giúp việc sử dụng trở nên thuận tiện hơn, phù hợp với nhiều đối tượng người dùng. Hệ thống cũng có thể

được cải thiện bằng cách thu thập thêm dữ liệu thực tế có độ đa dạng cao hơn, giúp mô hình thích ứng tốt hơn với các tình huống sử dụng ngoài đời. Ngoài ra, nhóm có thể nghiên cứu các kỹ thuật tối ưu hóa mô hình như pruning, quantization hoặc knowledge distillation để giảm kích thước và tăng tốc độ dự đoán. Cuối cùng, hướng phát triển nâng cao là mở rộng từ nhận dạng ký tự đơn lẻ sang nhận dạng chữ viết tay tự nhiên (HTR – Handwritten Text Recognition), cho phép xử lý cả từ, cụm từ hoặc câu hoàn chỉnh, từ đó đưa hệ thống đến gần hơn với các ứng dụng OCR thực tế.